# Ders 6 Çalışma Özeti

## Ders 6 Çalışma Özeti

### Genel Konular

- Dersin tarihi 13 Kasım 2020 Cuma. Ders Zoom üzerinden yapıldı.
  - Asıl anlatım yaklaşık 00:08:40'ta başlar. Kaydın başı bekleme ve selamlaşmadır.
  - Ders bir arayla iki bölüm. Ara yaklaşık 01:10-01:27 arasıdır.
  - Hocanın internet bağlantısı ikinci bölümde birkaç kez koptu.
- Geçen derste bağlantı sorunu yüzünden anlaşılmayan Warshall algoritmasının tekrarı.
- Bağıntıların matrisle gösterimi (matris bağıntıları).
- Fonksiyonlar
  - Fonksiyon tanımı, tanım ve değer kümesi.
  - Fonksiyonlarda toplama ve çarpma.
  - Birebir (one-to-one, injective), örten (onto, surjective) ve birebir-örten (bijective) fonksiyonlar.
  - Ters fonksiyon: liste, $ax + b$, $\frac{ax+b}{cx+d}$ ve ikinci dereceden polinom biçimleri.
  - Fonksiyon bileşkesi: birleşme özelliği var, değişme özelliği yok.
  - Üstel ve logaritmik fonksiyonlar.
  - Dizgi (string) ters çevirme.
  - Taban (floor) ve tavan (ceiling) fonksiyonları.
- Bölümün soru-cevap slaytları
  - Küme işlemleri, Venn şeması ile gölgelendirme.
  - Üç kümeli sayma problemi.
  - Doğru/yanlış ve karşı örnek (counterexample) soruları.
  - Dizi toplamları, sayı tabanları.
  - Bağıntı özellikleri, denklik bağıntısı, ters fonksiyon soruları.
- Yeni bölüm: Algoritmalar
  - Bu bölüm bu dönem derse yeni eklendi. Konu başlıkları: algoritma kavramı, temel veri yapıları, fonksiyonların büyümesi, asimptotik gösterim, karmaşıklık, özyinelemeli algoritmalar ve rekürans bağıntıları.
  - Bu derste: algoritma tanımı ve özellikleri, sözde kod (pseudocode), en büyük elemanı bulma.
  - Arama algoritmaları: doğrusal arama (linear search), ikili arama (binary search).
  - Sıralama algoritmaları: kabarcık sıralaması (bubble sort), araya ekleme sıralaması (insertion sort).
  - Öklid algoritması ile en büyük ortak bölen (EBOB).
  - Algoritma tasarım yöntemlerine kısa bakış.

### Hocanın Özellikle Vurguladığı Kısımlar

- Fonksiyonu fonksiyon yapan tek şart vardır: tanım kümesindeki her eleman değer kümesinde **tam olarak bir** elemana gider.
  - İki farklı eleman aynı değere gidebilir. Örnek: $x^2$'de $-2$ ve $2$ ikisi de $4$'e gider.
  - Bir eleman iki farklı değere gidemez.
- Bir fonksiyonun tersinin de fonksiyon olması için fonksiyon birebir ve örten (bijective) olmalıdır.
- Taban ve tavanı yuvarlama (round) ya da kesme (trunc) ile karıştırmayın.
  - $\lfloor 4.9 \rfloor = 4$ olur, 5 olmaz.

- Venn şeması ile sayma: bilgileri en içteki bölgeden başlayarak yerleştirin. Bütün bölgeler dolunca hangi soru gelirse gelsin cevap şemadan okunur.
- Algoritma çalışırken önce akış diyagramı ya da sözde kod yazın, sonra kodlayın.
  - Hoca bunun bölümün bir ekolü olduğunu, doğrudan koda geçmeyi başlangıçta doğru bulmadıklarını söyledi.
- Uç nokta (edge case) problemlerini mutlaka düşünün.
  - Bir algoritma çoğu girişte doğru çalışıp uç durumlarda hata verebilir.
- `exit` ve `goto` kullanmak "racona ters". Sınavda puan kırılabilir.
  - Arama bulunduğunda durması gereken döngülerde `for` yerine `while` kullanın.
- İkili arama yalnız **sıralı** dizide çalışır.
  - Küçük dizilerde fark milisaniyelerdir. Büyük dizilerde önce hızlı bir sıralama, sonra ikili arama daha verimlidir.
- Bir algoritmayı anlamak için küçük bir örnek üzerinde indislerin nasıl değiştiğini adım adım takip edin.
  - Hoca Warshall ve insertion sort için bunu özellikle önerdi.
  - Kitaplara giren algoritmalar alanında en verimli olanlardır. Kendi kodunuzla bunları karşılaştırın.
- Sınav hakkında: Sınavın türü henüz belli değil.
  - Algoritma sorusu sorulursa büyük ihtimalle sözde kodla sorulacak. Bu ders tam bir algoritma dersi değil.

## Kısa Tekrar Notları

- Fonksiyon: $f : X \to Y$, her $x \in X$ için tek bir $f(x) \in Y$.
- Birebir: $f(a) = f(b) \Rightarrow a = b$. Örnek: $2^x$ birebir, $x^2$ (reel sayılarda) değil.
- Örten: değer kümesinde açıkta eleman kalmaz.
- Birebir-örten (bijective): ikisi birden. Tersi de fonksiyondur.
- Ters fonksiyon kalıpları:
  - $f(x) = ax + b \Rightarrow f^{-1}(x) = \dfrac{x - b}{a}$
  - $f(x) = \dfrac{ax + b}{cx + d} \Rightarrow f^{-1}(x) = \dfrac{-dx + b}{cx - a}$
  - Polinomda $y = f(x)$ yazılır, $x$ yalnız bırakılır.
- Bileşke: $(f \circ g)(x) = f(g(x))$. Birleşme var: $f \circ (g \circ h) = (f \circ g) \circ h$. Değişme yok: genelde $f \circ g \neq g \circ f$.
- $f$ ve $g$ birbirinin tersiyse $f \circ g = g \circ f = $ birim fonksiyon. Örnek: $2^x$ ve $\log_2 x$.
- $\lfloor x \rfloor$: $x$'e eşit veya ondan küçük en büyük tam sayı. $\lceil x \rceil$: $x$'e eşit veya ondan büyük en küçük tam sayı.
- Algoritma: bir hesaplamayı yapmak ya da bir problemi çözmek için sonlu, kesin komutlar kümesi.
- Algoritmanın özellikleri: giriş, çıkış, doğruluk, sonluluk, etkililik, genellik.
- Doğrusal arama en kötü durumda $n$ karşılaştırma yapar. İkili arama aralığı her adımda yarıya indirir.
- Öklid: $\gcd(m, n) = \gcd(n, m \bmod n)$, $n = 0$ olunca cevap $m$.

## Detaylı Açıklamalar

### Warshall algoritmasının tekrarı

- Bağıntı $R = \{(b,a), (b,c), (c,a), (c,d), (d,c), (a,d)\}$ önce graf, sonra $a, b, c, d$ sırasıyla 0-1 matrisi olarak yazılır.
- Algoritma: $W_{ij} \leftarrow W_{ij} \vee (W_{ik} \wedge W_{kj})$, $k$ en dışta.
  - $W_{ik}$ ve $W_{kj}$ birlikte 1 ise $i \to k \to j$ zinciri vardır. Geçişme için $i \to j$ de olmalıdır.
  - VEYA işlemi, zaten var olan 1'i korur, eksikse ekler.
- Hoca $k = 1$, $i = 1$ için $j = 1$ ve $j = 4$ adımlarını tek tek gösterdi: $W_{11} = 0 \vee (0 \wedge 0) = 0$, $W_{14} = 1 \vee (0 \wedge 1) = 1$.
- Sonuç matrisinde $a$, $b$, $c$, $d$ satırlarının hepsi `1011` olur. Ara adımların tablosu ders 5 özetindedir.

- Hoca öğrenciyken geçişme kapanışı ödevini çok uzun bir kodla yaptığını anlattı. Warshall'ın tek satırlık işlemi bu yüzden değerli.

**Matris bağıntıları**

- Bağıntının satırları tanım kümesinin, sütunları değer kümesinin elemanlarıdır. $(x, y) \in R$ ise ilgili göz 1, değilse 0.
- Bu derste ayrıca yeni bir şey anlatılmadı. Hoca "matris bağıntılarında başka bir şey yok" dedi.

**Fonksiyon örnekleri**

- $\{(1, a), (2, b), (3, a)\}$: fonksiyondur.
- $\{(1, a), (2, b), (3, c), (1, b)\}$: fonksiyon değildir. 1 iki farklı elemana gidiyor.
- Slayttaki ok diyagramlarının değerlendirmesi:
  - Fonksiyon, birebir, örten değil (değer kümesinde açıkta eleman var).
  - Fonksiyon, birebir değil (iki eleman aynı değere gidiyor), örten.
  - Fonksiyon, birebir ve örten: birebir-örten.
  - Fonksiyon, ne birebir ne örten ($a$ ve $c$ aynı değere gidiyor, 4 açıkta).
  - Fonksiyon değil ($a$ iki elemana gidiyor). Diğer özelliklere bakmaya gerek yok.

**Ters fonksiyon**

- Liste biçimi: $f = \{(1, a), (2, c), (3, b)\} \Rightarrow f^{-1} = \{(a, 1), (c, 2), (b, 3)\}$.
- $f(x) = x + 1 \Rightarrow f^{-1}(x) = x - 1$.
- Kesirli biçim örneği: $f(x) = \dfrac{3x - 5}{x - 4} \Rightarrow f^{-1}(x) = \dfrac{4x - 5}{x - 3}$.
  - $a$ ile $d$ yer değiştirir ve işaretleri değişir. Python ile $f(f^{-1}(x)) = x$ olduğu kontrol edildi.
- Polinom örneği: $y = x^2 - 6x + 13$.
  - $y = (x - 3)^2 + 4 \Rightarrow y - 4 = (x - 3)^2 \Rightarrow x = \sqrt{y - 4} + 3$.
  - Yani $f^{-1}(x) = \sqrt{x - 4} + 3$.
- Soru-cevap slaytı:
  - $f(x) = 4x + 2 \Rightarrow f^{-1}(x) = \dfrac{x - 2}{4}$.
  - $f(x) = 3 + \dfrac{1}{x} \Rightarrow f^{-1}(x) = \dfrac{1}{x - 3}$.

**Fonksiyon bileşkesi**

- Slayttaki örnek: $g(x) = x^2 - 1$, $f(x) = 3x + 5$.
  - $f \circ g(x) = f(g(x)) = 3(x^2 - 1) + 5 = 3x^2 + 2$.
  - $g \circ f(x) = g(f(x)) = (3x + 5)^2 - 1 = 9x^2 + 30x + 24$.
  - İkisi eşit değil. Değişme özelliği yok.
- $f(x) = 2^x$, $g(x) = \log_2 x = \lg x$:
  - $f \circ g(x) = 2^{\lg x} = x$, $g \circ f(x) = \lg 2^x = x$.
  - Bu iki fonksiyon birbirinin tersidir.

**Dizgi (string) tersi**

- $\alpha = a_1 a_2 a_3 a_4$ dizgisinin ters çevrilmişi $a_4 a_3 a_2 a_1$'dir.
- Ters çevirmeyi iki kez uygulamak dizginin kendisini verir.

**Taban ve tavan**

- Slayttaki örnekler (Python ile doğrulandı):

| İfade | Sonuç | İfade | Sonuç |
|---|---|---|---|
| $\lfloor 8.3 \rfloor$ | 8 | $\lceil -8 \rceil$ | $-8$ |
| $\lfloor 1/2 \rfloor$ | 0 | $\lceil -1/2 \rceil$ | 0 |
| $\lfloor -1/2 \rfloor$ | $-1$ | $\lceil 1/2 \rceil$ | 1 |
| $\lfloor -8.7 \rfloor$ | $-9$ | $\lceil -11.3 \rceil$ | $-11$ |
| $\lfloor 3.1 \rfloor$ | 3 | $\lceil 6 \rceil$ | 6 |
| $\lfloor 7 \rfloor$ | 7 | $\lceil 3.1 \rceil$ | 4 |

- Negatif sayılarda dikkat: $\lfloor -8.7 \rfloor = -9$ çünkü $-9 < -8.7$.
- Yuvarlama (round): ondalık kısım $\geq 0.5$ ise yukarı. Kesme (trunc): ondalık kısım atılır.

**Küme soruları**

- $U = \{1, \ldots, 10\}$, $A = \{1, 4, 7, 10\}$, $B = \{1, 2, 3, 4, 5\}$, $C = \{2, 4, 6, 8\}$.
  - $A \cup B = \{1, 2, 3, 4, 5, 7, 10\}$
  - $B - A = \{2, 3, 5\}$
  - $\overline{U} = \emptyset$, $A \cup U = U$
  - $\overline{B} \cap (C - A) = \{6, 8\}$
  - $(A \cup B) - (C - B) = \{1, 2, 3, 4, 5, 7, 10\}$ ($C - B = \{6, 8\}$ ve $A \cup B$ ile kesişmiyor)
- Venn gölgelendirme: $((C \cap A) - \overline{(B - A)}) \cap C = \emptyset$.
  - $(C \cap A) - \overline{(B - A)} = (C \cap A) \cap (B - A)$. $B - A$ ile $A$ ayrık olduğu için sonuç boştur.
- Doğru/yanlış: "her $X$, $Y$ için $(X \cap Y) \cup (Y - X) = X$".
  - Sol taraf $Y$'ye eşittir. İfade yanlıştır. Karşı örnek: $X = \{1\}$, $Y = \{2\}$.
  - Hoca önce "doğru" dedi, sonra çizimi tekrar yapıp "yanlış" olarak düzeltti.

**Üç kümeli sayma problemi**

- 191 öğrenci. Fransızca (F) 65, işletme (B) 76, müzik (M) 63. F∩B 36, F∩M 20, B∩M 18, üçü birden 10.
- Bölgeler (içten dışa):
  - Üçü: 10
  - Yalnız F∩B: 26, yalnız F∩M: 10, yalnız B∩M: 8
  - Yalnız F: 19, yalnız B: 32, yalnız M: 35
- Soru: Fransızca ve müzik alıp işletme almayan kaç kişi? **10**.
- Hiç ders almayan: $191 - 140 = 51$ (derste sorulmadı, şemadan hesaplandı).

**Dizi toplamları ve sayı tabanları**

- $a_i = i^2 - 3i + 3$ için $\sum_{i=1}^{4} a_i = 1 + 1 + 3 + 7 = 12$.
- $y_i = 2^i - 1$, $z_i = i(i-1)$ için $\left(\sum_{i=1}^{3} y_i\right)\left(\sum_{i=1}^{3} z_i\right) = 11 \cdot 8 = 88$.
  - Hoca bunu $\sum y_i \cdot 8 = 8 + 24 + 56$ olarak hesapladı. Sonuç aynı.
- 9450 sayısı ikili ya da sekizli olamaz (9 rakamı var). Onlu ya da on altılı olabilir.
- 1101010 sayısı ikili, onlu ya da on altılı okunabilir. 10'dan farklı bir taban kastediliyorsa taban yazılmalıdır.

**Bağıntı soruları**

- $\{1, \ldots, 5\}$ üzerinde $x + y \leq 6$:
  - $R$ 15 çift içerir: $(1,1), \ldots, (1,5)$; $(2,1), \ldots, (2,4)$; $(3,1), (3,2), (3,3)$; $(4,1), (4,2)$; $(5,1)$.
  - "Non-reflexive" ($(1,1), (2,2), (3,3)$ var, $(4,4), (5,5)$ yok).
  - Simetrik (toplama değişmeli). Ters simetrik değil.
  - Geçişli değil: $(5,1)$ ve $(1,2)$ var, ama $(5,2)$ yok ($5 + 2 = 7$).
- "$R$ ve $S$ ters simetrikse $R \cup S$ de ters simetriktir": yanlış. Karşı örnek: $R = \{(1,2)\}$, $S = \{(2,1)\}$.

- $\{(1,1),\ldots,(5,5),(1,5),(5,1),(3,5),(5,3),(1,3),(3,1)\}$: denklik bağıntısıdır (doğru).
- "3 divides $x+y$", $\{1,\ldots,5\}$ üzerinde: denklik bağıntısı değildir (yanlış). $(1,1)$ yok, yansımalı değil.

**Algoritma kavramı**

- "Algoritma" sözcüğü El-Harezmi'nin (büyük olasılıkla MS 780-850) adından gelir. Kavram 20. yüzyılda hayatımıza girdi.
- Algoritma: bir hesaplamayı yapmak ya da bir problemi çözmek için sonlu ve kesin komutlar kümesi.
- En büyük elemanı bulma örneği: $3,5,10,1,15,7,2$.
  - İlk eleman geçici en büyük seçilir. Her eleman bununla karşılaştırılır, büyükse yeni en büyük olur. Sonuç 15.

```
procedure max(a1, a2, ..., an: integers)
max := a1
for i := 2 to n
    if max < ai then max := ai
return max
```

- Sözde kod: doğal dile ve programlama diline benzer ama ikisi de değildir. Hoca bunu bir dikiş kalıbına benzetti: aynı kalıptan farklı kumaşlarla (C, Java, Python) aynı model çıkar.
- Algoritmanın özellikleri (slayttan):
  - Giriş: belirli bir kümeden giriş değerleri.
  - Çıkış: girişlerden üretilen çıkış değerleri, yani çözüm.
  - Doğruluk: her giriş için doğru çıkış.
  - Sonluluk: her giriş için sınırlı sayıda adımdan sonra çıkış üretilir.
  - Etkililik: her adım doğru ve sınırlı sürede yapılabilmelidir.
  - Genellik: istenen formdaki bütün problemler için çalışır.
- Problem türleri: arama, sıralama, optimizasyon (en iyileme).

**Doğrusal arama**

```
procedure linear search(x: integer, a1, ..., an: distinct integers)
i := 1
while (i <= n and x != ai)
    i := i + 1
if i <= n then location := i
else location := 0
return location
```

- `for` + `if` ile de yazılabilir. Ama bulunduktan sonra döngü gereksiz yere sürer ya da `exit` gerekir. Bu yüzden `while` daha uygundur.

**İkili arama**

```
procedure binary search(x: integer, a1, ..., an: increasing integers)
i := 1
j := n
while i < j
    m := floor((i + j) / 2)
    if x > am then i := m + 1
    else j := m
if x = ai then location := i
else location := 0
return location
```

- Örnek: $i=1$, $j=10$ ise $m=\lfloor 11/2\rfloor = 5$. Taban fonksiyonu burada kullanılıyor.

**Kabarcık sıralaması (bubble sort)**

```
procedure bubble sort(a1, ..., an: real numbers with n >= 2)
for i := 1 to n - 1
    for j := 1 to n - i
        if aj > aj+1 then interchange aj and aj+1
```

- Her geçişte kalan en büyük eleman sona yerleşir. Bu yüzden iç döngü $n - i$'de biter.
- Örnek $3, 2, 4, 1, 5$:
  - $i = 1$ sonrası: $2, 3, 1, 4, 5$
  - $i = 2$ sonrası: $2, 1, 3, 4, 5$
  - $i = 3$ sonrası: $1, 2, 3, 4, 5$
  - $i = 4$: değişiklik yok.

**Araya ekleme sıralaması (insertion sort)**

```
procedure insertion sort(a1, ..., an: real numbers with n >= 2)
for j := 2 to n
    i := 1
    while aj > ai
        i := i + 1
    m := aj
    for k := 0 to j - i - 1
        a(j-k) := a(j-k-1)
    ai := m
```

- 2. elemandan başlar. $j$. eleman, ilk $j$ eleman arasında doğru yerine konur.
- Doğru yer doğrusal aramayla bulunur. Eleman `m`'de saklanır, aradaki elemanlar bir sağa kaydırılır.
- Örnek $3, 2, 4, 1, 5$: $2, 3, 4, 1, 5 \to 2, 3, 4, 1, 5 \to 1, 2, 3, 4, 5 \to 1, 2, 3, 4, 5$.

**Öklid algoritması**

```
procedure gcd(a, b: positive integers)
x := a
y := b
while y != 0
    r := x mod y
    x := y
    y := r
return x
```

- Örnek: $\gcd(60, 24)$: $60 \bmod 24 = 12$, $24 \bmod 12 = 0$. Sonuç 12.
- Slaytta "ardışık tam sayı kontrolü" ile EBOB bulan ikinci bir algoritma da var. Hoca bunu ödev olarak inceleyip Öklid ile farkını görmenizi istedi.

**Algoritma tasarım yöntemleri**

- Brute force, divide and conquer, decrease and conquer, transform and conquer, space-time tradeoff, greedy (açgözlü), dynamic programming, iterative improvement, backtracking, branch and bound.
- Görülen sıralamalar brute force'a yakındır. Quick sort bir divide and conquer algoritmasıdır.
- Ayrıntılar Algoritma Analizi dersinde işlenecek. Önümüzdeki iki hafta yürütme zamanı ve karmaşıklık işlenecek.

**Ders sonrası notlar**

- **Bileşke örneği:** Slaytta $g(x) = x^2 - 1$ var. Hoca $f \circ g$ için sözlü olarak "$3x^2 + 1$" dedi; slayttaki doğru sonuç $3x^2 + 2$. $g \circ f$ için tahtaya $(3x + 5)^2 + 1 = 9x^2 + 30x + 26$ yazdı. $g(x) = x^2 - 1$ olduğu için doğrusu $(3x + 5)^2 - 1 = 9x^2 + 30x + 24$. Python ile doğrulandı.

- **Üstel ve logaritmik fonksiyonlarda değişme:** Hoca "biri üstel biri logaritmikse $f \circ g = g \circ f$" dedi. Bu yalnız iki fonksiyon birbirinin tersiyse (aynı taban) doğrudur. Bir öğrencinin denediği $2^x$ ve $\log_3 x$ gibi farklı tabanlarda eşitlik bozulur. Hoca derste bunu kontrol edeceğini söyledi. Ayrıca $f \circ g(x) = x$ yalnız $x > 0$ için tanımlıdır.
- **Ters çevrilmiş dizgi:** Hoca "dizginin kendisiyle tersinin çarpımı boş kümedir" dedi ve "$\lambda$" yazdı. $\lambda$ boş küme değil, **boş dizgidir** (uzunluğu 0 olan dizgi). Ayrıca ters çevirme (reversal) işlemi, dizgiyle "çarpınca" boş dizgi veren bir ters eleman tanımlamaz.
- **Polinomun tersi:** $x^2 - 6x + 13$ reel sayılarda birebir değildir. Tersin fonksiyon olması için tanım kümesi $x \geq 3$ ile sınırlanmalıdır. Bu durumda $f^{-1}(x) = \sqrt{x-4} + 3$, $x \geq 4$.
- $f(x) = 3 + 1/x$ **tersi:** Hoca açıklamada bir an "$0.5x + 3$" gibi bir ifadeyle anlattı ve konu karıştı. Doğru çözüm: $y = 3 + 1/x \Rightarrow x = \dfrac{1}{y-3}$, yani $f^{-1}(x) = \dfrac{1}{x-3}$ (slayttaki cevap).
- **Devirli sayının tabanı:** Bir öğrenci $\lfloor 1.\overline{9} \rfloor$'u sordu. Matematikte $1.\overline{9} = 2$ olduğu için cevap **2**'dir. Hoca programlamada sayının sonlu basamakla saklandığını, bu yüzden 1 çıkabileceğini söyledi. Kayan noktalı sayılarda `1.9999999999999999` gibi bir yazım zaten 2.0'a yuvarlanır ve taban 2 verir. Sınavda matematiksel cevabı (2) verin.
- **"3 divides $x + y$" bağıntısı:** Hoca çiftleri $(1,2), (1,5), (2,4), (4,2), (5,1), (2,1)$ olarak yazdı. $(3,3)$ $(3+3=6)$, $(4,5)$ ve $(5,4)$ $(4+5=9)$ eksik kaldı. Doğru bağıntı 9 çifttir. $(1,1)$ olmadığı için sonuç ("denklik bağıntısı değil") yine doğrudur.
- **Sıralama bağıntısı tanımı:** Hoca bir yerde "simetrik, antisimetrik, geçişli ise sıralama bağıntısı" dedi. Doğrusu: **yansımalı**, ters simetrik ve geçişli.
- **Algoritmanın sonluluğu:** Hoca sonluluğu "sonsuz giriş verilemez" diye açıkladı. Slayttaki tanım doğrudur: algoritma her giriş için sınırlı sayıda adımdan sonra çıkış üretmelidir. Mesele girişin değil, adım sayısının sonlu olmasıdır.
- **Etkililik:** Hoca etkililiği "aynı işi 10 saniye yerine 10 dakikada yapmak kabul edilemez" diye anlattı. Bu verimlilik (efficiency) kavramıdır. Etkililik (effectiveness): her adımın kesin olarak ve sonlu sürede yapılabilmesidir.
- **Doğrusal arama izlemesi:** Derste $x = 14$ aranırken "$i = 15$ oldu, 15 14'ten küçük eşit mi" gibi bir karışıklık oldu. Döngüde $i$, aranan sayı $x$ ile değil, dizinin eleman sayısı $n$ ile karşılaştırılır.
- **El-Harezmi:** Hoca "Türk kökenli" dedi. El-Harezmi Harezm bölgesindendir (bugünkü Özbekistan-Türkmenistan). Kaynakların çoğu onu büyük olasılıkla İranlı (Fars) bir bilgin olarak tanıtır. Kökeni bu yüzden tartışmalıdır.
- **Sayma problemi:** Hoca bölgeleri yerleştirirken iki kez sayı karıştırdı ("Fransızca toplamı 76", "kalan 20 kişi"). Slayttaki Venn şemasındaki değerler (19, 26, 10, 10, 8, 32, 35) doğrudur ve verilerle tutarlıdır.
- Taban/tavan tablosu, küme işlemleri, toplamlar, ters fonksiyonlar, EBOB ve bağıntı özellikleri Python ile kontrol edildi.

**Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.